ROYAUME DU MAROC

UNIVERSITÉ ABDELMALEK ESSAÂDI

Faculté des Sciences et Techniques de Tanger (FSTT)

Département Génie Informatique — Cycle Ingénieur : Logiciels et Systèmes Intelligents (LSI)

MODULE JEE • RAPPORT D'ATELIER PRATIQUE

Application Web E-Commerce basée sur JSF 2.2 et JPA (EclipseLink / MySQL)

Conception orientée objet, mise en place de la couche de persistance transactionnelle JPA et développement des Managed Beans JSF pour la gestion d'une vitrine en ligne, du panier et des internautes.

Java JDK 11 | Maven | JSF 2.2 (Mojarra) | JPA 2.2 (EclipseLink 2.7.5) | MySQL 8 | Apache Tomcat 9

RÉALISÉ PAR

FIACH Hakima

Élève Ingénieure — Cycle LSI (FST Tanger)

ENCADRÉ PAR

Pr. Lotfi ELAACHAK

Professeur au Département Génie Informatique

Année Universitaire : 2026 / 2027

1. Introduction et Objectifs de l'Atelier

Dans le cadre du module JEE du cycle ingénieur LSI, cet atelier a pour objectif principal la maîtrise conjointe de l'API de persistance **JPA (Java Persistence API)** à travers l'implémentation **EclipseLink** et du framework web par composants **JSF (JavaServer Faces)**. Pour mettre en pratique ces concepts de manière concrète, j'ai conçu et réalisé de bout en bout l'application web e-commerce « **AtlasTech Store** », dédiée à la vente de matériel informatique et high-tech.

Le travail réalisé suit méthodiquement les quatre étapes demandées dans l'énoncé :

- **Étape 1 :** Élaboration d'un diagramme de classes UML complet centré sur la gestion de la Vitrine, du Panier et de l'Internaute, tout en intégrant le suivi des Commandes.
- **Étape 2 :** Création d'un projet Web dynamique (Servlet 4.0) géré par Maven et intégration des dépendances MySQL Connector, JPA/EclipseLink et JSF 2.2 dans le fichier `pom.xml`.
- **Étape 3 :** Développement de la couche modèle/persistance avec les entités JPA et la gestion explicite des transactions (`EntityTransaction`), accompagnée de la génération de la base de données relationnelle sous MySQL.
- **Étape 4 :** Mise en place des classes Managed Beans JSF pour chaque module métier ainsi que des vues Facelets XHTML associées.

2. Environnement Technique et Architecture 3-Tiers

L'application repose sur une architecture JEE en trois couches distinctes (Présentation, Métier/DAO, Persistance) déployée sur le conteneur de servlets **Apache Tomcat 9.0**. Ce découpage assure une séparation claire entre l'interface utilisateur JSF et les accès transactionnels à la base de données MySQL.

Composant / Outil	Version utilisée	Rôle au sein du projet
JDK & IDE	Eclipse Temurin JDK 11 / IntelliJ IDEA	Compilation du code source Java, gestion Maven intégrée et exécution locale via Smart Tomcat.
Apache Maven	Packaging `war`	Gestion centralisée du cycle de vie de build et résolution automatique des dépendances dans `pom.xml`.
Framework JSF	Mojarra 2.2.20	Contrôleur frontal `FacesServlet`, cycle de vie MVC, validation des formulaires et rendu des templates Facelets XHTML.
JPA / EclipseLink	JPA 2.2 / EclipseLink 2.7.5	Mapping Objet-Relationnel (ORM), génération automatique des tables DDL et gestion des transactions locales.
SGBD MySQL	MySQL / Connector-J 8.0	Stockage relationnel InnoDB dans la base `ecommerce_jsf_jpa` avec intégrité référentielle.
Serveur Web	Apache Tomcat 9.0.123	Conteneur Servlet 4.0 hébergeant l'application sous le contexte `/ecommerce-jsf-jpa` sur le port `8080`.

Le code source est organisé sous la racine de paquetages `ma.fstt.ecommerce` répartie en quatre sous-packages : `entities` (les 7 entités persistantes), `dao` (les classes d'accès aux données encapsulant les transactions JPA), `beans` (les Managed Beans JSF en session) et `util` (singleton `JPAUtil`, hachage SHA-256 des mots de passe, filtre UTF-8 et écouteur d'initialisation `AppInitListener`).

3. Étape 1 : Conception et Diagramme de Classes UML

Pour répondre fidèlement aux besoins fonctionnels d'un site e-commerce, le modèle métier s'articule autour de sept classes entités reliées par des associations JPA maîtrisées. L'accent a été mis sur l'interaction entre l'**Internaute**, la consultation de la **Vitrine** (structurée en catégories et produits) et la gestion persistante du **Panier** jusqu'à sa transformation en **Commande**.

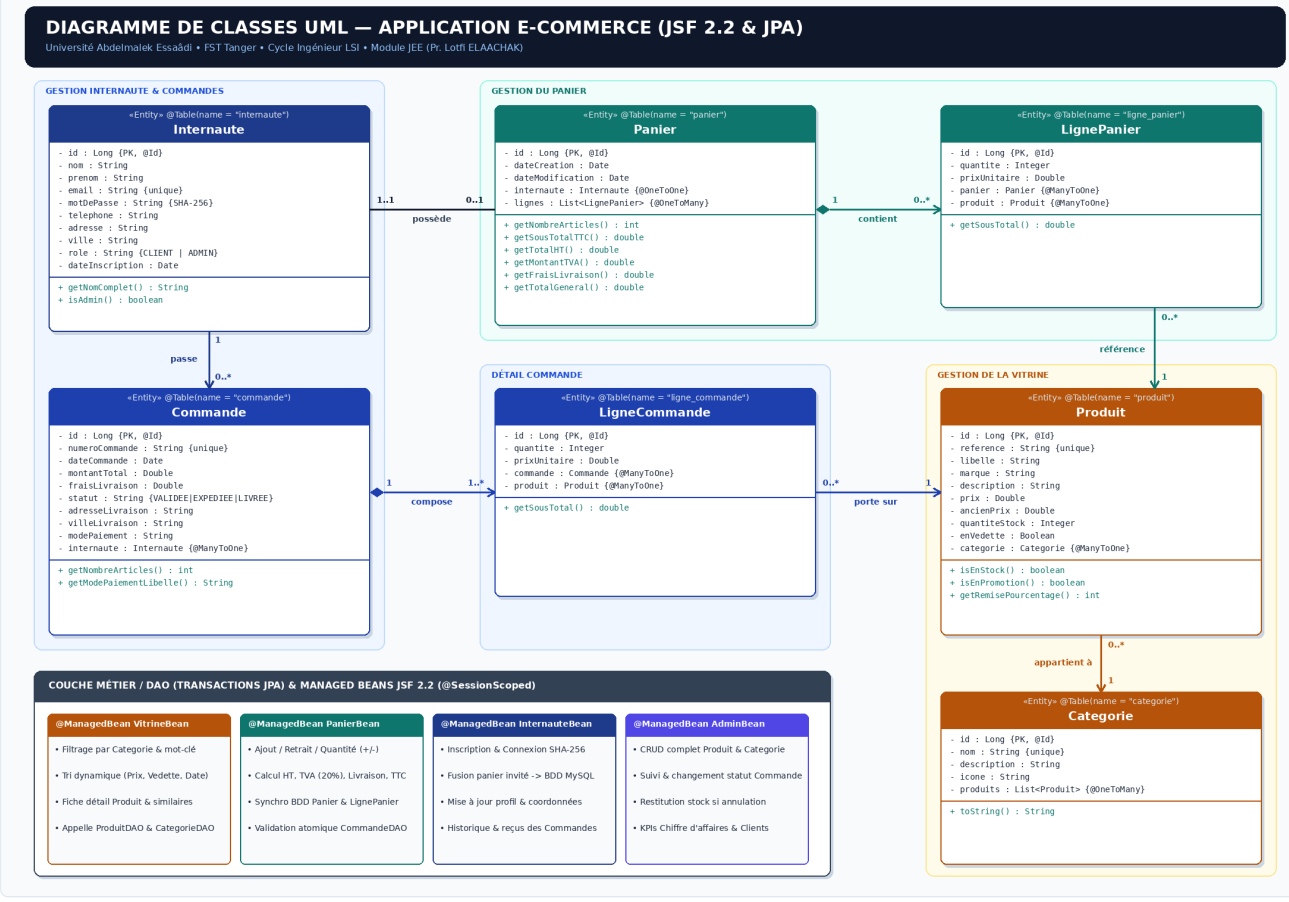


Figure 1 — Diagramme de classes UML de la couche d'entités JPA et des Managed Beans JSF associés.

Description des entités et des relations JPA

Entité JPA	Table MySQL	Attributs principaux & Associations ORM
Categorie	`categorie`	`id`, `nom` (unique), `description`, `icone`. Relation `@OneToMany` vers `Produit`.
Produit	`produit`	`id`, `reference`, `libelle`, `marque`, `prix`, `ancienPrix`, `quantiteStock`, `enVedette`. `@ManyToOne` vers `Categorie`.
Internaute	`internaute`	`id`, `nom`, `prenom`, `email` (unique), `motDePasse` (SHA-256), `ville`, `adresse`, `role` (CLIENT / ADMIN).
Panier	`panier`	`id`, `dateCreation`, `dateModification`. Lié en `@OneToOne` à `Internaute` et `@OneToMany(cascade=ALL, orphanRemoval=true)` à `LignePanier`.
LignePanier	`ligne_panier`	`id`, `quantite`, `prixUnitaire`. Associe un `Panier` et un `Produit` avec calcul du sous-total.
Commande	`commande`	`id`, `numeroCommande`, `dateCommande`, `montantTotal`, `statut`, `modePaiement`, `@ManyToOne Internaute`.
LigneCommande	`ligne_commande`	`id`, `quantite`, `prixUnitaire`. Figer le prix unitaire d'achat au moment de la validation de la commande.

4. Étape 2 : Configuration du Projet Web Maven

Le projet utilise un descripteur `WEB-INF/web.xml` configuré en version **Servlet 4.0**, où `javax.faces.webapp.FacesServlet` intercepte toutes les requêtes `*.xhtml` avec `index.xhtml` comme page d'accueil. Le fichier `pom.xml` intègre les dépendances nécessaires au fonctionnement conjoint de MySQL, EclipseLink et JSF 2.2 :

```
<dependencies>
    <!-- Connecteur JDBC MySQL -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.28</version>
    </dependency>
    <!-- API JPA & Implémentation EclipseLink -->
    <dependency>
        <groupId>org.eclipse.persistence</groupId>
        <artifactId>javax.persistence</artifactId>
        <version>2.2.1</version>
    </dependency>
    <dependency>
        <groupId>org.eclipse.persistence</groupId>
        <artifactId>eclipselink</artifactId>
        <version>2.7.5</version>
    </dependency>
    <!-- Framework JSF 2.2 (Mojarra API & Implémentation) -->
    <dependency>
        <groupId>com.sun.faces</groupId>
        <artifactId>jsf-api</artifactId>
        <version>2.2.20</version>
    </dependency>
    <dependency>
        <groupId>com.sun.faces</groupId>
        <artifactId>jsf-impl</artifactId>
        <version>2.2.20</version>
    </dependency>
    <!-- API Servlet 4.0 fournie par Apache Tomcat 9 -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>javax.servlet-api</artifactId>
        <version>4.0.0</version>
        <scope>provided</scope>
    </dependency>
</dependencies>
```

5. Étape 3 : Persistance JPA, Transactions et Base MySQL

Puisque Tomcat est un conteneur de servlets léger dépourvu de gestionnaire JTA intégré, l'unité de persistance `EcommercePU` déclarée dans `src/main/resources/META-INF/persistence.xml` utilise le mode `transaction-type="RESOURCE_LOCAL"`. La propriété `eclipselink.ddl-generation` positionnée à `create-or-extend-tables` permet à EclipseLink de générer automatiquement les 7 tables et leurs clés étrangères dans le schéma MySQL `ecommerce_jsf_jpa` dès le démarrage du serveur.

La classe utilitaire `JPAUtil` centralise l'unique instance d'`EntityManagerFactory`. Dans la couche DAO (`CategorieDAO`, `ProduitDAO`, `InternauteDAO`, `PanierDAO`, `CommandeDAO`), chaque opération d'écriture est encapsulée dans une transaction `EntityTransaction` garantissant les propriétés ACID. C'est notamment le cas lors de la validation du panier en commande :

```
EntityManager em = JPAUtil.getEntityManager();
EntityTransaction tx = em.getTransaction();
try {
    tx.begin();
    for (LignePanier lp : lignes) {
        Produit p = em.find(Produit.class, lp.getProduit().getId());
        if (p.getQuantiteStock() < lp.getQuantite()) {
            throw new IllegalStateException("Stock insuffisant pour : " + p.getLibelle());
        }
        p.setQuantiteStock(p.getQuantiteStock() - lp.getQuantite()); // Décrémentation atomique
        commande.getLignesCommande().add(new LigneCommande(commande, p, lp.getQuantite(), p.getPrix()));
    }
    em.persist(commande);
    // Vidage simultané du panier persistant de l'internaute dans la même transaction
    tx.commit();
} catch (RuntimeException e) {
    if (tx.isActive()) tx.rollback();
    throw e;
} finally {
    em.close();
}
```

Initialisation automatique de la BDD : Au démarrage de Tomcat, le listener `@WebListener AppInitListener` vérifie si la base est vide et insère automatiquement 5 catégories, 12 produits high-tech, un compte client et un compte administrateur, en complément du script exporté `sql/ecommerce_jsf_jpa.sql`.

6. Étape 4 : Managed Beans JSF et Interfaces Réalisées

Conformément à l'étape 4 de l'atelier, chaque volet fonctionnel de l'application est pris en charge par un Managed Bean JSF dédié (annoté avec `@ManagedBean` et `@SessionScoped`), relié aux pages Facelets XHTML via des expressions EL (Expression Language) :

Managed Bean	Vues XHTML associées	Responsabilités fonctionnelles
VitrineBean	`index.xhtml`, `produit.xhtml`	Chargement des catégories et produits, filtrage dynamique, recherche par mot-clé, tri par prix/nouveautés et affichage de la fiche détail avec articles similaires.
PanierBean	`panier.xhtml`, `commande-confirmation.xhtml`	Ajout d'articles avec vérification du stock, incrémentation/décrémentation des quantités, calcul HT / TVA (20%) / livraison gratuite dès 1 000 DH, synchronisation MySQL et validation de commande.
InternauteBean	`connexion.xhtml`, `inscription.xhtml`, `profil.xhtml`	Création de compte, authentification sécurisée, fusion automatique du panier invité lors de la connexion, modification des coordonnées et consultation de l'historique des commandes.
AdminBean	`admin.xhtml`	Back-office administrateur : indicateurs clés (chiffre d'affaires, compteurs), CRUD complet des produits et catégories, suivi et mise à jour des statuts de commandes.

Aperçu de l'application en exécution sous Apache Tomcat 9

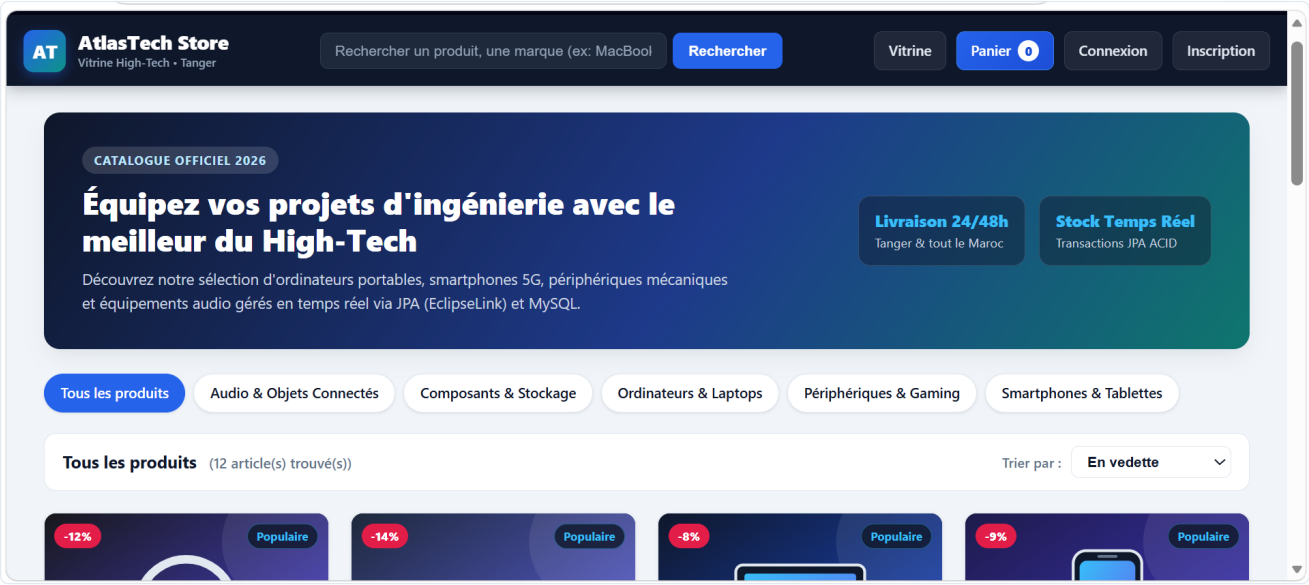


Figure 2 — Page d'accueil de la Vitrine (*index.xhtml*) : barre de recherche, filtres par catégorie et catalogue chargé depuis MySQL.

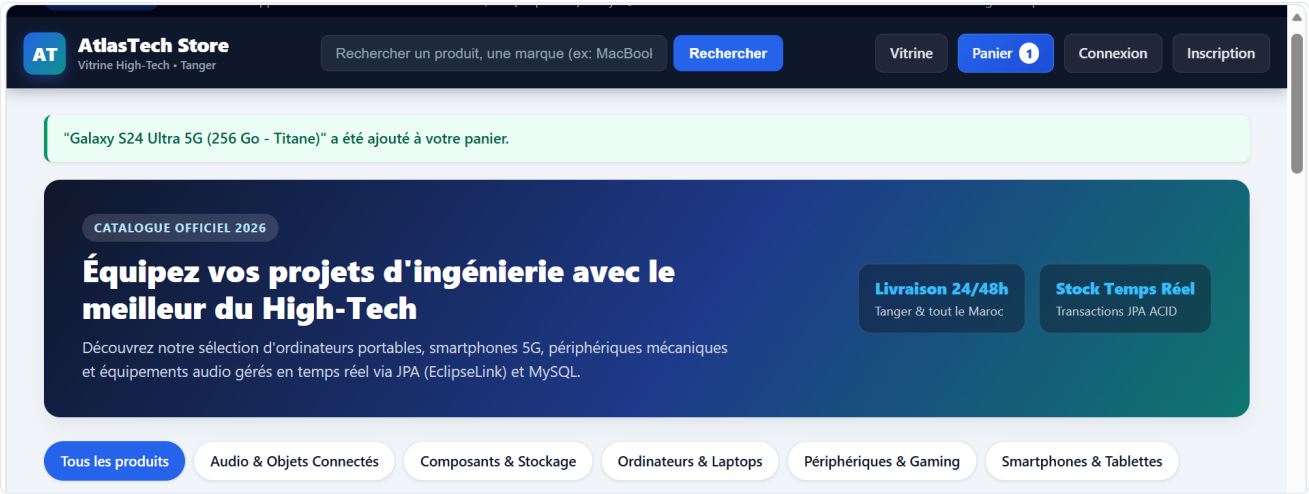


Figure 3 — Ajout d'un produit au panier depuis la vitrine : message de confirmation JSF et incrémentation dynamique du badge « Panier 1 ».

7. Gestion du Panier et de l'Espace Internaute

L'interface du panier permet à l'internaute de contrôler ses articles avant validation. Lorsqu'un visiteur non connecté ajoute des articles, ceux-ci sont conservés en session puis automatiquement fusionnés dans les tables `panier` et `ligne_panier` de MySQL dès son authentification ou son inscription.



Figure 4 — Vue de gestion du panier (*panier.xhtml*) avant ajout d'articles, avec redirection rapide vers la vitrine.

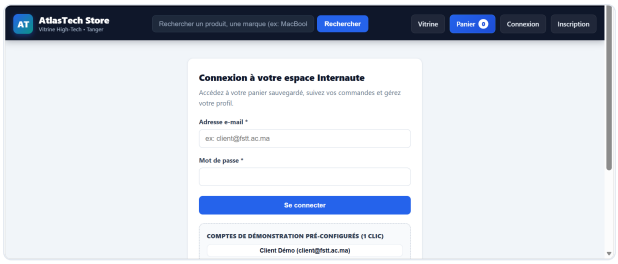


Figure 5 — Page d'authentification de l'internaute (*connexion.xhtml*) et accès rapide aux comptes de test.

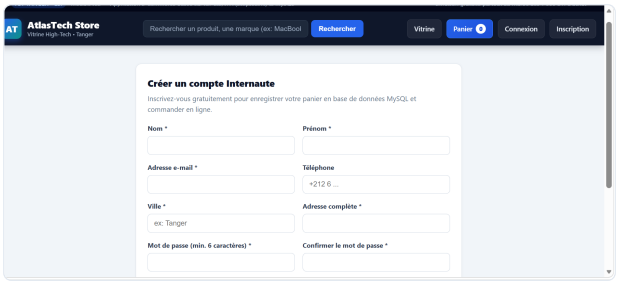


Figure 6 — Formulaire d'inscription d'un nouvel internaute (*inscription.xhtml*) avec contrôles de saisie JSF.

8. Conclusion

La réalisation de cet atelier m'a permis d'appréhender concrètement l'intégration entre **JSF 2.2** et **JPA (EclipseLink)** au sein d'une application web déployée sur **Apache Tomcat 9** et connectée à **MySQL**. Sur le plan de la persistance, la modélisation des associations entre entités et l'utilisation rigoureuse des transactions `EntityTransaction` m'ont permis de garantir la cohérence des données, en particulier lors du passage d'une commande qui impacte simultanément les commandes, le panier et les stocks produits.

Sur le plan de la couche présentation, le découpage en Managed Beans spécialisés (`VitrineBean`, `PanierBean`, `InternauteBean` et `AdminBean`) associé aux templates Facelets XHTML a grandement facilité la structuration du code et la réactivité de l'interface utilisateur. L'application obtenue est fonctionnelle, complète et directement évolutive vers de nouvelles briques JEE.